FRD

Functional Requirements Document (FRD)

AWS Glue ETL Pipeline for Customer Order Analytics

•

1. Document Overview

Purpose

This Functional Requirements Document defines the functional behavior of an AWS Glue-based ETL pipeline that ingests customer and order data from S3, applies data quality transformations, implements Slowly Changing Dimension Type 2 (SCD2) logic using Apache Hudi, and produces analytical aggregates for business intelligence purposes.

Scope

This FRD covers:

- Ingestion of customer and order datasets from S3
- Data cleaning and deduplication
- Registration of datasets in AWS Glue Data Catalog
- SCD Type 2 implementation for order summary tracking
- Incremental processing logic triggered by customer updates
- Generation of customer aggregate spend analytics
- Storage of curated and analytics datasets in S3

This FRD does NOT cover:

- Infrastructure provisioning or IAM policies
- Scheduling or orchestration mechanisms
- Data visualization or reporting layers
- Real-time streaming requirements

Assumptions

- AWS Glue jobs will have appropriate IAM permissions to read from source S3 paths and write to target S3 paths
- Source CSV files are well-formed and use standard delimiters
- The Glue Data Catalog database `gen_ai_poc_databrickscoe` already exists
- Apache Hudi libraries are available in the Glue environment
- "Null" (string) and NULL (actual null) are the only null representations requiring removal
- Date fields in source data are in a parseable format

• CustId is the join key between customer and order datasets

• The term "change" in SCD2 context refers to any modification in customer or order attributes

• Athena will be used for querying cataloged tables

• CloudWatch will be configured for job monitoring

• Lake Formation governance is optional and not functionally required

•

2. Functional Requirements

FR-INGEST-001: Ingest Customer Data from S3

• Title: Load Customer CSV Files from S3

• Description:

The system shall read customer data in CSV format from the specified S3 location and load it into a Glue DynamicFrame for processing.

• Business Objective:

Enable the ETL pipeline to access customer master data for downstream transformations and analytics.

• Inputs:

• Source: `s3://adif-sdlc/sdlc_wizard/customerdata/`

• Format: CSV

• Schema:

• `CustId` (data type not specified in BRD)

• `Name` (data type not specified in BRD)

• `EmailId` (data type not specified in BRD)

• `Region` (data type not specified in BRD)

• Processing Rules / Functional Steps:

1. Connect to S3 path `s3://adif-sdlc/sdlc_wizard/customerdata/`

2. Read all CSV files in the path

3. Parse CSV structure according to the defined schema

4. Load data into a Glue DynamicFrame

• Outputs:

• In-memory DynamicFrame containing raw customer records

• Preconditions:

• S3 path exists and contains at least one CSV file

• Glue job has read permissions on the S3 path

• Postconditions:

• Customer data is available in memory for cleaning and transformation

• Error / Validation Conditions:

• If S3 path does not exist, job must fail with clear error message

• If CSV files are malformed or cannot be parsed, job must fail with file-level error details

• If schema does not match expected columns, job must fail with schema mismatch error

•

FR-INGEST-002: Ingest Order Data from S3

• Title: Load Order CSV Files from S3

• Description:

The system shall read order transaction data in CSV format from the specified S3 location and load it into a Glue DynamicFrame for processing.

• Business Objective:

Enable the ETL pipeline to access order transaction data for join operations, SCD2 tracking, and spend analytics.

• Inputs:

• Source: `s3://adif-sdlc/sdlc_wizard/orderdata/`

• Format: CSV

• Schema:

• `OrderId` (data type not specified in BRD)

• `ItemName` (data type not specified in BRD)

• `PricePerUnit` (data type not specified in BRD)

• `Qty` (data type not specified in BRD)

• `Date` (data type not specified in BRD)

• `CustId` (data type not specified in BRD)

• Processing Rules / Functional Steps:

1. Connect to S3 path `s3://adif-sdlc/sdlc_wizard/orderdata/`

2. Read all CSV files in the path

3. Parse CSV structure according to the defined schema

4. Load data into a Glue DynamicFrame

• Outputs:

• In-memory DynamicFrame containing raw order records

• Preconditions:

• S3 path exists and contains at least one CSV file

• Glue job has read permissions on the S3 path

• Postconditions:

• Order data is available in memory for cleaning and transformation

• Error / Validation Conditions:

• If S3 path does not exist, job must fail with clear error message

• If CSV files are malformed or cannot be parsed, job must fail with file-level error details

• If schema does not match expected columns, job must fail with schema mismatch error

•

FR-CLEAN-001: Remove Null Values from Customer Dataset

• Title: Clean Null Values in Customer Data

• Description:

The system shall remove all records from the customer dataset that contain the string "Null" or actual NULL values in any field.

• Business Objective:

Ensure data quality by eliminating incomplete customer records that cannot support reliable analytics or joins.

• Inputs:

• Raw customer DynamicFrame from FR-INGEST-001

• Processing Rules / Functional Steps:

1. Scan all fields in each customer record

2. Identify records where any field contains the string "Null" (case-sensitive) or NULL value

3. Remove identified records from the dataset

4. Retain only records with complete, non-null values in all fields

• Outputs:

• Cleaned customer DynamicFrame with no null values

• Preconditions:

• Customer data has been successfully ingested

• Postconditions:

• All remaining customer records have valid, non-null values in all fields

• Error / Validation Conditions:

• If all records are removed due to null values, log a warning but do not fail the job

•

FR-CLEAN-002: Remove Null Values from Order Dataset

• Title: Clean Null Values in Order Data

• Description:

The system shall remove all records from the order dataset that contain the string "Null" or actual NULL values in any field.

• Business Objective:

Ensure data quality by eliminating incomplete order records that cannot support reliable analytics or joins.

• Inputs:

• Raw order DynamicFrame from FR-INGEST-002

• Processing Rules / Functional Steps:

1. Scan all fields in each order record

2. Identify records where any field contains the string "Null" (case-sensitive) or NULL value

3. Remove identified records from the dataset

4. Retain only records with complete, non-null values in all fields

• Outputs:

• Cleaned order DynamicFrame with no null values

• Preconditions:

• Order data has been successfully ingested

• Postconditions:

• All remaining order records have valid, non-null values in all fields

• Error / Validation Conditions:

• If all records are removed due to null values, log a warning but do not fail the job

•

FR-CLEAN-003: Remove Duplicate Customer Records

- Title: Deduplicate Customer Dataset

- Description:

The system shall identify and remove duplicate records from the customer dataset, retaining only unique customer entries.

- Business Objective:

Prevent duplicate customer records from causing incorrect join results and inflated analytics.

- Inputs:
- Null-cleaned customer DynamicFrame from FR-CLEAN-001

- Processing Rules / Functional Steps:

1. Identify duplicate records based on all customer fields
2. Retain one instance of each unique record
3. Remove all other duplicate instances

- Outputs:
- Deduplicated customer DynamicFrame

- Preconditions:
- Customer data has been cleaned of null values

- Postconditions:
- Customer dataset contains only unique records

- Error / Validation Conditions:
- If no records remain after deduplication, log a warning but do not fail the job

-

FR-CLEAN-004: Remove Duplicate Order Records

- Title: Deduplicate Order Dataset

- Description:

The system shall identify and remove duplicate records from the order dataset, retaining only unique order entries.

- Business Objective:

Prevent duplicate order records from causing incorrect aggregations and inflated spend calculations.

• Inputs:

• Null-cleaned order DynamicFrame from FR-CLEAN-002

• Processing Rules / Functional Steps:

1. Identify duplicate records based on all order fields

2. Retain one instance of each unique record

3. Remove all other duplicate instances

• Outputs:

• Deduplicated order DynamicFrame

• Preconditions:

• Order data has been cleaned of null values

• Postconditions:

• Order dataset contains only unique records

• Error / Validation Conditions:

• If no records remain after deduplication, log a warning but do not fail the job

•

FR-CATALOG-001: Register Customer Table in Glue Data Catalog

• Title: Catalog Cleaned Customer Data

• Description:

The system shall write the cleaned and deduplicated customer data to S3 and register it as a table in the AWS Glue Data Catalog.

• Business Objective:

Make customer data discoverable and queryable via Athena and other AWS analytics services.

• Inputs:

• Deduplicated customer DynamicFrame from FR-CLEAN-003

• Processing Rules / Functional Steps:

1. Write customer data to S3 in a suitable format (Parquet recommended)

2. Register the dataset in Glue Data Catalog database `gen_ai_poc_databrickscoe`

3. Create or update table `sdlc_wizard_customer`

4. Ensure table metadata reflects the customer schema

• Outputs:

• Glue Catalog table: `gen_ai_poc_databrickscoe.sdlc_wizard_customer`

• Persisted customer data in S3

• Preconditions:

• Customer data has been cleaned and deduplicated

• Glue Data Catalog database `gen_ai_poc_databrickscoe` exists

• Glue job has write permissions to target S3 location and Glue Catalog

• Postconditions:

• Customer table is queryable via Athena

• Table metadata is current and accurate

• Error / Validation Conditions:

• If database does not exist, job must fail with clear error message

• If write to S3 fails, job must fail and not update catalog

• If catalog registration fails, job must fail with catalog error details

•

FR-CATALOG-002: Register Order Table in Glue Data Catalog

• Title: Catalog Cleaned Order Data

• Description:

The system shall write the cleaned and deduplicated order data to S3 and register it as a table in the AWS Glue Data Catalog.

• Business Objective:

Make order data discoverable and queryable via Athena and other AWS analytics services.

• Inputs:

• Deduplicated order DynamicFrame from FR-CLEAN-004

• Processing Rules / Functional Steps:

1. Write order data to S3 in a suitable format (Parquet recommended)

2. Register the dataset in Glue Data Catalog database `gen_ai_poc_databrickscoe`

3. Create or update table `sdlc_wizard_order`

4. Ensure table metadata reflects the order schema

• Outputs:

• Glue Catalog table: `gen_ai_poc_databrickscoe.sdlc_wizard_order`

• Persisted order data in S3

• Preconditions:

• Order data has been cleaned and deduplicated

• Glue Data Catalog database `gen_ai_poc_databrickscoe` exists

• Glue job has write permissions to target S3 location and Glue Catalog

• Postconditions:

• Order table is queryable via Athena

• Table metadata is current and accurate

• Error / Validation Conditions:

• If database does not exist, job must fail with clear error message

• If write to S3 fails, job must fail and not update catalog

• If catalog registration fails, job must fail with catalog error details

•

FR-SCD2-001: Implement SCD Type 2 for Order Summary

• Title: Maintain Historical Order Summary with SCD Type 2

• Description:

The system shall join customer and order datasets, apply Slowly Changing Dimension Type 2 logic to track historical changes, and persist the result as a Hudi table in S3.

• Business Objective:

Provide a historical view of order summaries that tracks changes over time, enabling time-based analytics and audit trails.

• Inputs:

• Cleaned customer data from FR-CLEAN-003

• Cleaned order data from FR-CLEAN-004

• Existing `ordersummary` Hudi table (if present) from `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

• Processing Rules / Functional Steps:

1. Join customer and order datasets on `CustId`

2. For each joined record, compare with existing records in `ordersummary` table

3. If a record represents a change (any attribute differs from the current active record):

• Set `IsActive = False` for the existing active record

• Set `EndDate` to current timestamp for the existing active record

• Insert a new record with `IsActive = True`

• Set `StartDate` to current timestamp for the new record

• Set `EndDate` to NULL or a far-future date for the new record

• Update `OpTs` (operation timestamp) to current timestamp

4. If a record is new (no matching key in existing table):

• Insert as a new active record with `IsActive = True`, `StartDate = current timestamp`, `EndDate = NULL`

5. If a record is unchanged, no action is required

6. Write the updated dataset to S3 using Hudi format at `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

• Outputs:

• Hudi table: `ordersummary` at `s3://adif-sdlc/curated/sdlc_wizard/ordersummary/`

• Glue Catalog table: `gen_ai_poc_databrickscoe.ordersummary`

• Schema includes: joined customer and order fields plus `IsActive`, `StartDate`, `EndDate`, `OpTs`

• Preconditions:

• Customer and order data have been cleaned and are available

• S3 target path is accessible

• Hudi libraries are available in Glue environment

• Glue Data Catalog database exists

• Postconditions:

• All changes are tracked with historical versions

• Only one active record exists per unique business key

• Historical records are preserved with closed date ranges

• Error / Validation Conditions:

• If join produces no records, log a warning but complete the job

• If Hudi write fails, job must fail with Hudi error details

• If multiple active records exist for the same key after processing, job must fail with data integrity error

•

FR-INCR-001: Trigger SCD2 Updates on Customer Changes

• Title: Incremental Processing for Customer Updates

• Description:

The system shall detect changes in customer data and trigger corresponding SCD Type 2 updates in the `ordersummary` table.

• Business Objective:

Ensure that changes to customer master data are reflected in the order summary history without requiring full reprocessing.

• Inputs:

• New or updated customer records from FR-INGEST-001 and FR-CLEAN-003

• Existing `ordersummary` Hudi table

• Processing Rules / Functional Steps:

1. Identify customer records that are new or have changed since the last processing run

2. For each changed customer, identify all related records in the `ordersummary` table (by `CustId`)

3. Apply SCD Type 2 logic as defined in FR-SCD2-001 to update affected order summary records

4. Maintain `IsActive`, `StartDate`, `EndDate`, and `OpTs` fields according to SCD2 rules

• Outputs:

• Updated `ordersummary` Hudi table with new historical versions for affected records

• Preconditions:

• Customer data has been ingested and cleaned

• `ordersummary` table exists

• Change detection mechanism is in place (e.g., watermark, timestamp comparison)

• Postconditions:

• All order summary records related to changed customers reflect the updates

• Historical versions are preserved

• Error / Validation Conditions:

• If change detection fails, job must fail with detection error details

• If no changes are detected, job completes successfully with no updates

• If SCD2 update fails, job must fail as per FR-SCD2-001 error conditions

•

FR-AGG-001: Generate Customer Aggregate Spend Table

• Title: Calculate and Persist Customer Total Spend

• Description:

The system shall aggregate order data by customer to calculate total spend and persist the result as an analytics table in S3.

• Business Objective:

Provide a summarized view of customer spending for business intelligence and reporting purposes.

• Inputs:

• Cleaned and joined customer-order data (from FR-SCD2-001 processing, using active records only)

• Processing Rules / Functional Steps:

1. Filter `ordersummary` table to include only active records (`IsActive = True`)

2. Calculate `TotalAmount` for each order as `PricePerUnit Qty`

3. Group records by customer `Name` and `Date`

4. Sum `TotalAmount` for each group

5. Write the aggregated result to S3 at `s3://adif-sdlc/analytics/customeraggregatespend/`

6. Register the result in Glue Data Catalog as table `customeraggregatespend`

• Outputs:

• Analytics table: `customeraggregatespend` at `s3://adif-sdlc/analytics/customeraggregatespend/`

• Glue Catalog table: `gen_ai_poc_databrickscoe.customeraggregatespend`

• Schema:

• `Name` (data type not specified in BRD)

• `TotalAmount` (data type not specified in BRD)

• `Date` (data type not specified in BRD)

• Preconditions:

• `ordersummary` table exists and contains active records

• S3 target path is accessible

• Glue Data Catalog database exists

• Postconditions:

• Customer aggregate spend data is available for querying via Athena

• Table reflects the latest aggregated spend by customer and date

• Error / Validation Conditions:

• If no active records exist in `ordersummary`, create an empty table and log a warning

• If aggregation fails, job must fail with aggregation error details

• If write to S3 or catalog registration fails, job must fail with appropriate error message

•

3. Non-Functional Considerations

Monitoring and Observability

• AWS CloudWatch shall be used to monitor Glue job execution, log errors, and track performance metrics

• Job logs must be retained for troubleshooting and audit purposes

Data Governance (Optional)

• AWS Lake Formation may be used to enforce fine-grained access control and data governance policies

• If implemented, Lake Formation permissions must align with IAM policies

Query Performance

• Athena shall be used to query cataloged tables

• Data formats and partitioning strategies should be optimized for Athena query performance

Scalability

• The solution must handle growing data volumes without manual intervention

• Glue jobs should be configured with appropriate DPU (Data Processing Unit) allocations

•

4. Requirement Traceability Notes

	Requirement ID	BRD Section	Description	
	---	---	---	
	FR-INGEST-001	Section 1	Maps to "Customer CSV: s3://adif-sdlc/sdlc_wizard/customerdata/"	
	FR-INGEST-002	Section 1	Maps to "Order CSV: s3://adif-sdlc/sdlc_wizard/orderdata/"	
	FR-CLEAN-001	Section 3	Maps to "Remove 'Null' and NULL values" for customer data	
	FR-CLEAN-002	Section 3	Maps to "Remove 'Null' and NULL values" for order data	
	FR-CLEAN-003	Section 3	Maps to "Remove duplicates in each dataset" for customer data	
	FR-CLEAN-004	Section 3	Maps to "Remove duplicates in each dataset" for order data	
	FR-CATALOG-001	Section 4	Maps to "Tables: sdlc_wizard_customer"	
	FR-CATALOG-002	Section 4	Maps to "Tables: sdlc_wizard_order"	
	FR-SCD2-001	Section 5	Maps to "SCD Type 2 (Hudi on S3)" for ordersummary table	
	FR-INCR-001	Section 6	Maps to "Incremental Logic" for customer updates	
	FR-AGG-001	Section 7	Maps to "Aggregation Table: customeraggregatespend"	

• End of Functional Requirements Document